Javascript on the Frontend

Tobias Schulz-Hess

Duration: 2 hours
Prerequisite: Basic JavaScript, HTML & CSS knowledge

JavaScript on the Frontend

What You'll Learn Today

- 🌐 **DOM Manipulation** - Selecting and modifying elements
- 🎨 **CSS Integration** - Dynamic styling with JavaScript
- ⚡ **Event Handling** - Responding to user interactions
- 📝 **Form Processing** - Reading and validating user input
- 🔄 **Fetch API** - Communicating with backend services
- 🛠️ **CRUD Operations** - Complete data management cycle

This builds on your existing JavaScript knowledge

How to embed JavaScript in the frontend?

Plain

```
1  <script>
2    console.log("Hello world!");
3  </script>
```

Warning
not recommended!

Loading from file

```
1  <script src="js/main.js"></script>
```

Better do this at the end of your file, just before `</body>`. This way you can make sure, everything is loaded, when you want to access it in your js code.

Key Differences

Backend JavaScript

- File system access
- Database connections
- Server-side APIs
- Node.js modules

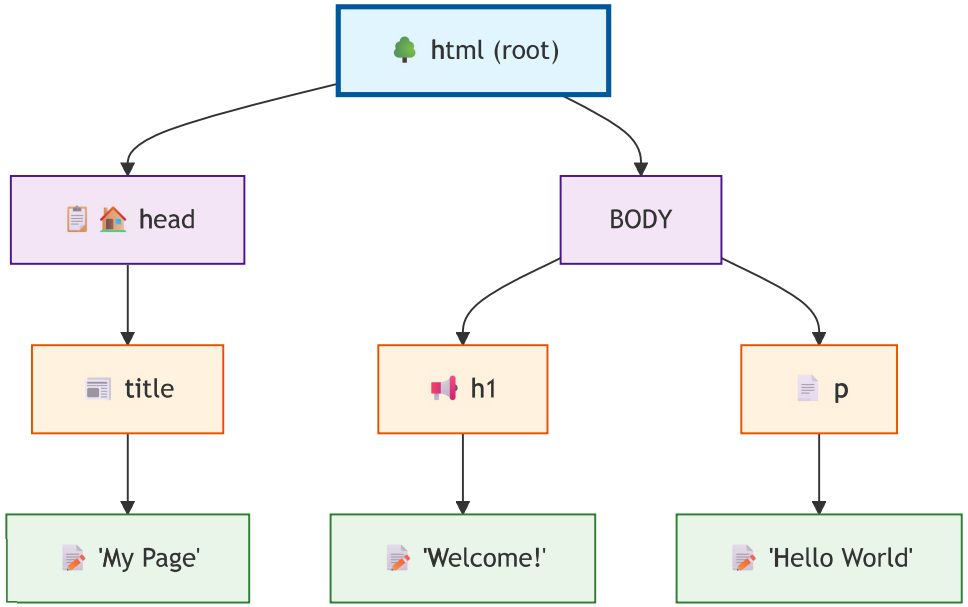
Frontend JavaScript

- DOM manipulation
- Browser APIs
- User interactions
- Client-side validation

DOM-Tree

The **Document Object Model (DOM)** represents HTML as a hierarchical tree structure:

- Every HTML element, attribute, and text becomes a **node**
- Nodes have **parent-child relationships**
- The `<html>` tag is the **root node**
- Text content forms **text nodes**



The DOM bridges the gap between HTML markup and JavaScript manipulation!

DOM Selection Methods

Let's start with the fundamentals:

```
// Select by ID
const header = document.getElementById('main-header');

// Select by class (first match)
const button = document.querySelector('.btn-primary');

// Select multiple elements
const allButtons = document.querySelectorAll('.btn');
```

Query-Selector

```
h1 { /* Accessing all h1 tags */ }

#header { /* Accessing element with id header */ }

.quotes { /* Accessing all elements with class quotes */ }

p.quotes { /* Accessing all p tags with class quotes */ }

body p { /* Accessing all descendants (Descendant combinator) */ }

body > p { /* Accessing direct children (Child combinator) */ }

h1 + p { /* h1 tag directly followed by first p tag (Next Adjacent sibling combinator) */ }

h1 ~ p { /* h1 tag directly followed by any p tag (General sibling combinator) */ }

h1, h2, h3 { /* Accessing all h1, h2 and h3 tags */ }
```

Form Data Access

Multiple ways to access form values:

```
// Direct element access
const email = document.getElementById('email').value;
const password = document.getElementById('password').value;

// Using form reference
const form = document.forms.loginForm;
const userData = form.email.value;

// FormData API (modern approach)
const formData = new FormData(form);
const email = formData.get('email');
```

🧪 Lab: Complete Registration Form

Build a registration form with validation:

1. **Fields:** First name, last name, email, password, confirm password

2. **Validation Rules:**

- All fields required
- Email format validation
- Password minimum 8 characters
- Passwords must match

3. **Features:**

- Real-time validation feedback
- Error messages with styling
- Success message on valid submission

Fetch API Overview

The **Fetch API** provides a modern interface for making HTTP requests:

```
// Basic GET request
fetch('/api/users')
.then(response => response.json())
.then(data => console.log(data));

// Using async/await (preferred)
const response = await fetch('/api/users');
const data = await response.json();

// Error handling
const data = await response.json().catch(err => console.error(err));
```

Remember: Fetch returns Promises - use async/await or .then()

Live Coding Example: Basic Fetch Operations

CRUD Operations with Fetch

Create, **R**ead, **U**pdate, **D**elete - the four basic operations

```
8   const getItems = async () => {
9   const response = await fetch('/api/items');
10  return await response.json();
11  };
12
13  // UPDATE (PUT)
14  const updateItem = async (id, data) => {
15  const response = await fetch(/api/items/${id}, {
16  method: 'PUT', headers: {'Content-Type': 'application/json'}, body: JSON.stringify(data)
17  });
18
19  // DELETE (DELETE)
20  const deleteItem = async (id) => {
21  const response = await fetch(/api/items/${id}, { method: 'DELETE' });
22  return response.ok;
23  };
```

🧪 Final Lab: Todo Application

Build a complete CRUD todo application:

1. **HTML Structure:** Form to add todos, list to display them
2. **Create:** Add new todos via form submission
3. **Read:** Display all todos, show individual todo details
4. **Update:** Mark todos as complete/incomplete, edit todo text
5. **Delete:** Remove todos from the list

API Endpoints to use:

- GET `/api/todos` - Get all todos
- POST `/api/todos` - Create new todo
- PUT `/api/todos/:id` - Update todo
- DELETE `/api/todos/:id` - Delete todo

Use the json-server package to get some backend server or use your project backend, if you have it already.

What You've Learned Today

✅ Core Concepts Mastered

- **DOM Manipulation** - Selecting, modifying, and styling elements
- **Event Handling** - Responding to user interactions effectively
- **Form Processing** - Reading and validating user input
- **Fetch API** - Making HTTP requests to backend services
- **CRUD Operations** - Complete data management workflow

📚 Additional Resources

- MDN Web Docs
- JavaScript.info
- W3Schools JavaScript Tutorial

Thank You!

Questions & Discussion

You're now ready to build dynamic frontend applications with JavaScript!

*Remember: The best way to learn is by building projec